

Serviço Nacional de Aprendizagem Industrial - SENAI DF
Brasília, 12 de junho de 2026
Anderson de Matos Guimarães
Professor: Rômulo



DESENVOLVEDOR WEB COM PYTHON E DJANGO

Exercício 01

Considerando a aula de 11 de junho e o contexto do nosso projeto, crie uma nova classe de entidade chamada **tecnologia**.

- Classe EixoTecnologia
 - Id (gerado pelo Django)
 - nome
- MakeMigrations no aplicativo
- Migrate no aplicativo

Resolução:

Em motorartigos > models.py:

```
class EixoTecnologia(models.Model):  
    nome = models.CharField(max_length=100)  
  
    def __str__(self):  
        return self.nome
```

Depois, no terminal:

1. MakeMigrations

```
python manage.py makemigrations motorartigos
```

2. Migrate

```
python manage.py migrate motorartigos
```

1 Configurações 'internas' da entidade

1.1 Personalizar nome das entidades (caso queira excluir o padrão django: nomeapp_nomeentidade)

Em motorartigos > models.py, dentro de cada entidade, após `def __str__`:

```
class EixoTecnologia(models.Model):
    nome = models.CharField(max_length=100)

    def __str__(self):
        return self.nome

# Inserir padrão de nomenclatura de tabelas:
class Meta:
    db_table = "eixo"
```

Após, no terminal, MakeMigrations e Migrate:

```
python manage.py makemigrations motorartigos

python manage.py migrate motorartigos
```

2 Trabalhando com Django Admin (administra todos os apps)

No terminal:

1. MakeMigrations:

```
python manage.py makemigrations # aqui não entra o nome do aplicativo, já que
queremos entrar no django admin e não no app
```

2. Migrate:

```
python manage.py migrate
```

3. Após, verificar no MySQL Workbench consultar todas as tabelas criadas:

```
SHOW TABLES;
```

Aqui, além das tabelas criadas no app, aparecerão novas tabelas de **admin**.

4. Criar o **Super Usuário**:

```
python manage.py createsuperuser
```

Será solicitado:

- Nome Anderson
- E-mail anderson.m.guimaraes@icloud.com
- Senha senha padrão AG
- Confirmação de senha

No terminal, rodar o servidor:

```
python manage.py runserver
```

Após, na barra de endereço do navegador:

<http://127.0.0.1:8000/admin>

Será solicitado o usuário e a senha

4.1 Configurar o admin

Em motorartigos > admin.py

```
from django.contrib import admin # já vem dentro de admin.py
from .models import Autor, EixoTecnologia # Importar as Entidades criadas

# Register your models here.

class AutorAdmin(admin.ModelAdmin):
    list_display = ('nome', 'biografia', 'email',)
    search_fields = ('nome',)

admin.site.register(Autor, AutorAdmin)
```

Após, voltar no navegador <http://127.0.0.1:8000/admin>:

- Atualizar a página
- Verificar que o campo do app foi criado (**motorartigos**)
- Clicar em **Autors**
- Clicar em **ADICIONAR AUTOR**

Obs.: Por enquanto, apenas o **superuser** poderá cadastrar novos autores.

Exercício 02

Crie o admin do EixoTecnologia

1. Em motorartigos > admin.py, no fim do arquivo:

```
class EixoTecnologiaAdmin(admin.ModelAdmin):
    list_display = ('nome',)
    search_fields = ('nome',)
```

```
admin.site.register(EixoTecnologia, EixoTecnologiaAdmin)
```

2. Atualizar a página admin no navegador e verificar que a nova entidade **EixoTecnologia** foi criada.
3. Fazer as inclusões que julgar necessárias.

5 Variáveis de ambiente

- Na raiz do diretório, criar o arquivo `.env`
- Colocar a SECRET_KEY localizada em setup/settings.py

Obs.: no `.env` não precisa ter as aspas simples do `SECRET_KEY`

- Na SECRET_KEY do settings.py, deixar a chave entre aspas vazia.
- Instalar a biblioteca **dotenv**:
 - No terminal: `pip install python-dotenv`

Observações:

1. Como várias bibliotecas já foram instaladas, criar o requirements.txt via freeze:

- No terminal:
 - `pip freeze > requirements.txt`

Será criado um arquivo na raiz do projeto chamado **requirements.txt** contendo todas as bibliotecas que foram instaladas até o momento.

2. Quando o repositório for clonado, para baixar todas as bibliotecas de uma vez:

- No terminal:
 - `pip install -r requirements.txt`

3. Criar o `.gitignore`

- Na raiz do projeto criar o arquivo `.gitignore` com o seguinte conteúdo:
 - `venv`
 - `.venv`
 - `.env`

4. Importações necessárias no arquivo setup/settings.py:

```
from pathlib import Path, os # acrescentado o 'os' para configurações do sistema
from dotenv import load_dotenv # acrescentado

load_dotenv() # cria a função do dotenv
```

5. Informar a chave (`SECRET_KEY`) em settings.py:

```
# SECURITY WARNING: keep the secret key used in production secret!  
SECRET_KEY = str(os.getenv('SECRET_KEY')) # o 'os' vai no arquivo .env e pega a  
SECRET_KEY (a mesma que salvamos)
```

Após essas configurações, basta rodar o servidor:

No terminal:

```
python manage.py runserver
```

Exercício 03

A empresa solicitou mais uma modificação no sistema. Eles estão necessitando de um novo aplicativo. O nome do aplicativo é **chatbot**.

Sua missão: criar o aplicativo e instalar o aplicativo e fazer o foguetinho aparecer normalmente.

- Resolução:

1. No terminal, criar o novo app **chatbot**:

```
python manage.py startapp chatbot
```

2. No terinal, rodar o servidor:

```
python manage.py runserver
```

PARABÉNS: seu foguetinho decolou